

TaxWijs

Microservices Architecture Guide

A complete explanation + step-by-step migration plan

What You Will Learn in This Document:

1. What microservices are and how they differ from a monolith
2. The real benefits -- and the real costs -- of microservices
3. Core concepts: API Gateway, service discovery, message queues
4. How TaxWijs is currently built (monolith architecture)
5. What TaxWijs would look like as microservices (7 services)
6. Step-by-step migration plan with realistic time estimates
7. What tools you would need: Kubernetes, Docker Swarm, RabbitMQ
8. Inter-service communication: REST vs gRPC vs events
9. The honest recommendation: when to migrate and when NOT to

Version 1.0 | May 2026 | For: TaxWijs Engineering

Confidential -- Internal Use Only

Table of Contents

Section 1	Monolith vs Microservices -- The Big Picture
Section 2	What Exactly Are Microservices?
Section 3	The Benefits of Microservices
Section 4	The Real Costs and Challenges
Section 5	Core Concepts You Must Understand
Section 6	Inter-Service Communication
Section 7	Container Orchestration: Kubernetes vs Docker Swarm
Section 8	TaxWijs Current Architecture (Monolith Analysis)
Section 9	TaxWijs as Microservices (Proposed Architecture)
Section 10	The 7 Services: What Each Does
Section 11	Step-by-Step Migration Plan
Section 12	Infrastructure You Would Need
Section 13	The Honest Recommendation
Appendix A	Microservices Glossary

[INFO] Estimated reading time: 25-35 minutes. This document is designed to give you a complete, honest picture -- including why microservices might NOT be the right choice right now.

Section 1 -- Monolith vs Microservices: The Big Picture

Every software application starts as a monolith. A monolith is a single application where all the code runs together in one process. As an application grows, teams sometimes split it into microservices -- many small, independent applications that each do one specific job and communicate with each other over a network.

Neither approach is universally better. The best choice depends on team size, traffic volume, deployment frequency, and technical maturity. Understanding the trade-offs is essential before making any decision.

What a Monolith Looks Like

MONOLITH (What TaxWijs Is Today)

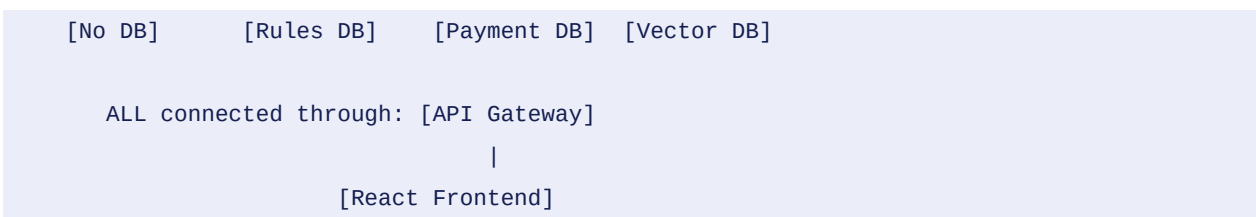
```
+-----+
|           ONE Django Application           |
|                                           |
| [Users] [Auth] [Chat] [Calculator]         |
| [Admin] [Tax Rules] [Payments] [RAG]       |
|                                           |
| All sharing:                             |
| * One database (PostgreSQL)               |
| * One web server (Django/Gunicorn)        |
| * One Docker container                    |
| * One deployment pipeline                 |
| * One codebase                            |
+-----+
|           |           |
| [PostgreSQL] [React Frontend]
```

What Microservices Look Like

MICROSERVICES (What TaxWijs Could Become)

```
[Auth Service]  [User Service]  [Chat Service]
|              |              |
| [User DB]    | [User DB]    | [Chat DB]

[Calculator]    [Tax Rules]    [Payments]    [RAG/Search]
|              |              |              |
```



Side-by-Side Comparison

Monolith (Current TaxWijs)	Microservices (Potential Future)
One codebase, easy to navigate	Many repos, each team owns one service
One deployment command	Each service deploys independently
Easy local development	Complex local setup (run 7+ services)
Simple debugging (one log stream)	Logs scattered across many services
One database, simple queries	Each service has its own DB
All features share the same memory	Services are isolated from each other
If it crashes, everything crashes	One service crash does not take all down
Hard to scale individual parts	Scale only the parts under load
All code must use same language	Each service can use different language
Perfect for small teams	Requires large teams to benefit

Section 2 -- What Exactly Are Microservices?

A microservice is a small, independently deployable application that does exactly one thing. It has its own database, its own web server, its own deployment pipeline, and its own codebase. It exposes an API (usually HTTP/REST or gRPC) that other services can call.

The Defining Principles of Microservices

Principle 1: Single Responsibility

Each service does ONE job and does it well. A chat service handles conversations. An auth service handles login and tokens. A calculator service computes tax numbers. They never mix. If you find a service doing two unrelated things, split it.

Principle 2: Own Your Data

This is the hardest principle to follow. Each service must have its OWN database. Services are NEVER allowed to share a database or read each other's tables directly. If the Chat service needs user information, it calls the User service's API -- it does NOT connect to the User service's database directly.

[WARN] This 'database per service' rule is what makes microservices hard. Every cross-service data need becomes an API call, and every API call can fail. You trade simplicity for isolation.

Principle 3: Independent Deployability

You must be able to deploy the Chat service without touching any other service. This means no shared code (no shared library that if changed, breaks everything). Services communicate only through stable, versioned APIs.

Principle 4: Designed for Failure

In a microservices system, any service can fail at any time. The system must be designed so that when one service fails, the rest keep working in a degraded state. For example: if the RAG search service goes down, the calculator should still work. This requires circuit breakers, timeouts, and fallback logic.

Principle 5: Built Around Business Capabilities

Services are NOT split by technical layer (database layer, API layer, UI layer). They are split by business function. 'Auth', 'Payments', 'Tax Calculation', 'Document Search' are business capabilities. Each becomes a service.

Real-World Example: Netflix

Netflix is the most famous microservices company. They have over 700 microservices. When you press Play, it calls: the User service (who are you?), the Streaming service (which server is closest?), the Recommendation service (log this watch), the Billing service (is subscription active?), and more -- all in parallel, all within milliseconds. This scale is why they chose microservices. They have 8,000 engineers.

[TIP] Netflix, Amazon, Uber, and Google are often cited as microservices success stories. They ALL had large engineering teams (hundreds to thousands) before migrating. The architecture was a consequence of their scale, not the cause of it.

Section 3 -- The Benefits of Microservices

Benefit 1: Independent Scaling

In a monolith, if your tax calculator gets a traffic spike on March 31st (IB deadline), you must scale the entire application -- including parts that are idle. With microservices, you scale ONLY the calculator service. Run 10 instances of the calculator, 1 instance of everything else. This saves money and improves performance.

```
# Docker Swarm: scale just the calculator
docker service scale taxwijs_calculator=10
docker service scale taxwijs_auth=2
docker service scale taxwijs_chat=5

# Kubernetes:
kubectl scale deployment calculator --replicas=10
kubectl scale deployment auth --replicas=2
```

Benefit 2: Independent Deployment

In a monolith, to fix a bug in the chat feature, you redeploy the entire application -- which means 30-60 seconds of downtime for the calculator, auth, and everything else. With microservices, you redeploy only the chat service. The rest keep running. Zero downtime for unrelated features.

Benefit 3: Technology Freedom

Each service can use the best tool for its job. The calculator could be written in Rust for maximum speed. The RAG search service could be Python because it uses ML libraries. The auth service could be Go for low latency. They just need to agree on the API contract.

Benefit 4: Team Autonomy

Large organizations split teams around services. The 'Payments team' owns the Payments service end-to-end. They deploy on their own schedule, make their own architectural decisions, and are accountable for their service's uptime. No coordination meetings needed to deploy.

Benefit 5: Fault Isolation

If the RAG search service has a memory leak and crashes, only search is affected. Auth, calculator, and

chat keep running. In a monolith, a memory leak anywhere can take down the whole application.

Benefit 6: Easier to Understand (Per Service)

A developer joining the Chat service team only needs to understand the chat codebase -- a few hundred lines. They don't need to understand tax calculation or payment processing. Cognitive load is reduced per team member.

[WARN] These benefits are REAL but they require scale to matter. Independent deployment matters when you deploy 50 times a day. Independent scaling matters when traffic varies by 100x between services. Team autonomy matters when you have 5+ teams working simultaneously.

Section 4 -- The Real Costs and Challenges

Most articles about microservices focus on the benefits. This section gives you the honest picture of what microservices COST. These are not theoretical -- they are real operational burdens that catch teams off guard.

Cost 1: Distributed System Complexity

When a user sends a chat message in TaxWijs, the monolith handles it in one function call. In microservices, it becomes: Auth service (verify token) -> Chat service (receive message) -> RAG service (search relevant rules) -> Calculator service (if numbers needed) -> Chat service (stream response). That is 4 network calls. Each can fail. Each adds latency. Each needs a timeout and retry.

[WARN] A function call takes nanoseconds. A network call takes 1-100 milliseconds. A monolith that queries its own database is 100-1000x faster than a microservice that calls another service for the same data.

Cost 2: Operational Overhead

With a monolith you manage 3 things: Django, PostgreSQL, nginx. With 7 microservices you manage: 7 separate applications, 7 separate databases, 7 separate deployment pipelines, 7 separate log streams, 7 separate health checks, 1 API Gateway, 1 message broker (RabbitMQ/Kafka), 1 service registry, 1 container orchestrator (Kubernetes), 1 centralized logging system, 1 distributed tracing system, 1 centralized secrets manager.

That is roughly 25 components vs 3. Every component can fail. Every component needs monitoring.

Cost 3: Data Consistency Nightmares

In a monolith, you wrap multiple database operations in a single transaction. Either all succeed or all fail. Simple. In microservices, there is no global transaction across service databases. If the Payments service records a payment but the User service fails to upgrade the subscription, the user is charged but gets no access. Fixing this requires implementing 'Saga pattern' (a sequence of compensating transactions) -- a significant engineering effort.

Cost 4: Testing Is Much Harder

In a monolith, a single integration test can verify the entire checkout flow. In microservices, you need: unit tests per service, integration tests per service, contract tests between every pair of services that communicate, end-to-end tests that run all services together, and chaos engineering tests (what happens

when auth goes down mid-checkout?).

Cost 5: Debugging Is Distributed

In a monolith, a user reports an error. You look at one log file. You find the error. In microservices, an error message like 'calculation failed' could originate in any of 3 services that were involved in that request. You need distributed tracing (tools like Jaeger or Zipkin) to follow a request across service boundaries. This is additional infrastructure and additional learning curve.

Cost 6: Network Failures Are Now Your Problem

In a monolith, if the calculator function fails, you handle the exception. In microservices, the Calculator service might: not respond (timeout), respond with an error (500), respond slowly (latency spike), or respond correctly but the response gets lost in transit. Your Chat service must handle all four cases gracefully.

Summary: What Microservices Cost

Challenge	Monolith	Microservices
Deployment complexity	1 command	Kubernetes + CI/CD per service
Database management	1 database	7+ databases
Error tracking	1 log file	Distributed tracing required
Developer onboarding	Learn one app	Learn API contracts for all services
Local development	docker compose up	Run 7+ services + gateway + broker
Transaction management	DB transaction	Saga pattern (complex)
Testing scope	Integration tests	+ Contract tests + Chaos tests
Infrastructure cost	1 VPS	Kubernetes cluster (3+ nodes)

Section 5 -- Core Concepts You Must Understand

5.1 -- API Gateway

The API Gateway is the single entry point for all client requests. The React frontend never talks to individual services directly. It only talks to the Gateway. The Gateway routes each request to the correct service.

The Gateway also handles:

- * Authentication -- verifying JWT tokens so individual services don't have to
- * Rate limiting -- blocking clients who send too many requests
- * Load balancing -- distributing requests across multiple instances of a service
- * SSL termination -- handling HTTPS so services can use plain HTTP internally
- * Request logging -- one place to log all incoming requests
- * Response caching -- caching frequent responses to reduce service load

Popular API Gateway tools: Kong, AWS API Gateway, Traefik, nginx (with Lua), APISIX.

[INFO] Without an API Gateway, your frontend would need to know the addresses of every service. Every service address change would require a frontend update. The Gateway hides this complexity.

5.2 -- Service Discovery

In a microservices system, services need to find each other. You cannot hardcode IP addresses because services move around (especially in Kubernetes). Service discovery solves this: each service registers itself with a 'registry', and other services look up addresses from the registry.

There are two types:

- * Client-side discovery: The calling service queries the registry and picks an instance
- * Server-side discovery: The load balancer queries the registry (Kubernetes does this automatically)

In Kubernetes, service discovery is built-in. Each service gets a DNS name like 'calculator.default.svc.cluster.local' that always resolves to a healthy instance.

5.3 -- Message Brokers (Async Communication)

Not all communication between services needs to be synchronous (request/response). Some events can be published to a queue and consumed asynchronously. A message broker (RabbitMQ, Apache Kafka, AWS SQS) handles this.

Example: When a user subscribes to Premium:

1. Payments service: processes payment -> publishes event 'payment.succeeded'
2. Message broker: receives the event, stores it
3. User service: reads the event -> upgrades user to Premium
4. Email service: reads the event -> sends welcome email
5. Analytics service: reads the event -> logs the conversion

The Payments service does NOT wait for steps 3-5. It published and moved on.
Steps 3-5 happen independently, in parallel, without coupling.

RabbitMQ = good for task queues, simple routing. Kafka = good for high-volume event streaming, replay capability.

5.4 -- Circuit Breaker

A circuit breaker prevents one failing service from taking down everything else. If the RAG service is slow or failing, the Chat service should stop calling it and return a fallback response, rather than waiting forever and queuing up thousands of requests.

States: Closed (normal), Open (service failing -- stop calling), Half-Open (testing if service recovered).

Libraries: Resilience4j (Java), Hystrix (Java/deprecated), tenacity (Python), or built into service mesh (Istio).

5.5 -- Service Mesh

A service mesh (Istio, Linkerd) adds a 'sidecar proxy' to every service. This proxy handles: mTLS encryption between services, circuit breaking, retries, tracing, and traffic management -- all without changing application code. This is advanced infrastructure used by large organizations.

5.6 -- Distributed Tracing

When a request touches 4 services, you need to trace its path across all of them. Distributed tracing assigns a 'trace ID' to each incoming request. Every service passes this ID to the next service. A tracing system (Jaeger, Zipkin, DataDog APM) collects all spans and shows you a timeline of where the request spent time.

Section 6 -- Inter-Service Communication

Three Styles of Communication

Style	When to Use	Technology	Pros	Cons
Synchronous	Simple CRUD, real-time	HTTP/JSON	Simple, universal, easy	Caller waits,
Synchronous	High-performance internal	Protocol Buffers over	Fast, typed, streaming	More setup,
Async Events	Background jobs, notifications, fan-out	RabbitMQ / Kafka	Decoupled, resilient, scalable	Harder to trace, eventual consistency

REST vs gRPC in Detail

REST (Representational State Transfer) uses HTTP and JSON. It is the most common API style. Every developer knows it. Easy to test with curl or Postman. Slight performance overhead due to JSON parsing.

gRPC uses Protocol Buffers (a binary format) over HTTP/2. It is 5-10x faster than REST for the same operation because binary encoding is smaller and faster. It also supports server-side streaming (the server sends multiple responses) and bidirectional streaming (both send simultaneously). The trade-off: you need to define .proto files, generate code, and learn the gRPC toolchain.

For TaxWijs: REST for everything initially. gRPC only if calculator performance becomes a bottleneck.

The Saga Pattern for Distributed Transactions

When an operation spans multiple services (e.g., subscribe to Premium: charge card AND upgrade user), you cannot use a database transaction. Instead, you use the Saga pattern.

Two types of Saga:

- * Choreography: Each service publishes events and reacts to events from others. No central coordinator. Simpler but harder to trace.
- * Orchestration: A central 'Saga Orchestrator' service calls each service in sequence, and if one fails, it calls compensating actions (refund, rollback, etc.).

Orchestrated Saga -- Subscribe to Premium:

Saga Orchestrator:

Step 1 -> Call Payments: charge EUR 19.99

[OK] -> Step 2 -> Call User: set plan=premium

[OK] -> Step 3 -> Call Email: send confirmation

[FAIL at Step 2] -> Call Payments: refund EUR 19.99 (compensating action)

[FAIL at Step 3] -> log error, email will retry later (non-critical)

Section 7 -- Container Orchestration: Kubernetes vs Docker Swarm

With 7 services running, you need something to manage starting them, restarting crashed ones, scaling them up and down, and distributing traffic. That is what container orchestration does.

Option A: Docker Swarm

Docker Swarm extends Docker Compose to work across multiple servers. You already know Docker Compose -- Swarm uses the same file format with minor additions. It is simpler than Kubernetes and can run on 1-5 servers comfortably.

- * Pro: Same docker-compose.yml syntax -- minimal learning curve
- * Pro: Works on 1 server, scales to ~10 servers
- * Pro: Simple to set up: docker swarm init
- * Pro: Built into Docker -- no extra tools to install
- * Con: Limited auto-scaling (no CPU-based auto-scale)
- * Con: Less ecosystem than Kubernetes
- * Con: Docker Inc has deprioritized Swarm in favor of Kubernetes

```
# Initialize swarm:
docker swarm init

# Deploy all services:
docker stack deploy -c docker-compose.yml taxwijs

# Scale a service:
docker service scale taxwijs_calculator=5

# View all services:
docker service ls
```

Option B: Kubernetes (K8s)

Kubernetes is the industry standard for container orchestration. It is dramatically more powerful than Docker Swarm but also much more complex. It was originally built at Google and open-sourced in 2014. Today, every major cloud provider (AWS EKS, Google GKE, Azure AKS) offers managed Kubernetes.

- * Pro: Extremely powerful auto-scaling (CPU, memory, custom metrics)
- * Pro: Self-healing: automatically restarts crashed containers
- * Pro: Rolling deployments with zero downtime

- * Pro: Huge ecosystem: Helm charts, operators, service meshes
- * Pro: Industry standard -- many managed options
- * Con: Very steep learning curve (new concepts: Pods, Deployments, Services, Ingress, etc.)
- * Con: Minimum 3-node cluster recommended (3 VPS servers at ~EUR 20/mo each)
- * Con: Requires DevOps knowledge to operate safely
- * Con: Complex YAML configuration files

```
# Key Kubernetes concepts:
# Pod: one or more containers that run together
# Deployment: manages a set of identical Pods
# Service: stable network endpoint for a set of Pods
# Ingress: routes external traffic to Services
# ConfigMap: configuration data
# Secret: sensitive data (passwords, API keys)

# Deploy calculator service:
kubectl apply -f calculator-deployment.yaml
kubectl apply -f calculator-service.yaml

# Scale it:
kubectl scale deployment calculator --replicas=5

# Check status:
kubectl get pods
kubectl get services
```

Recommendation for TaxWijs

Current Size	Team Size	Recommended Orchestration
1-3 microservices	1-2 developers	Docker Compose (what you have now)
3-7 microservices	2-5 developers	Docker Swarm (simple, low overhead)
7+ microservices	5+ developers	Kubernetes (managed: EKS, GKE, or DigitalOcean)

[TIP] If you decide to migrate TaxWijs to microservices, START with Docker Swarm. It handles 7 services easily and the learning curve is much lower. Migrate to Kubernetes only when Swarm becomes a bottleneck.

Section 8 -- TaxWijs Current Architecture (Monolith Analysis)

Before designing a microservices architecture, you must understand the current architecture thoroughly. Here is a complete analysis of TaxWijs as it stands today.

Current Application Structure

```

nl-tax-ai/
  backend/                                <- Single Django 6 application
    apps/
      users/                             <- Auth, registration, profiles
      chat/                              <- AI chat, conversations, SSE streaming
      tax/                               <- Tax rules, RAG retrieval, IB fields
      calculator/                        <- Tax calculation engine
      admin/                             <- Admin panel APIs
    config/
      settings.py                        <- All configuration in one place
      authentication.py                  <- Shared JWT authentication
  frontend/                              <- React 19 SPA
    src/pages/                           <- Landing, Chat, Calculator, Admin...
  phase1/                                <- Tax knowledge base (data)
  phase2/                                <- RAG pipeline (ChromaDB, embeddings)
  docker-compose.yml                     <- 3 containers: db, backend, frontend

```

What Modules Currently Communicate

From	To	How	What For
Chat (views.py)	Tax (retriever.py)	Direct Python	RAG search for context
Chat (views.py)	Calculator (views.py)	Direct Python	Tax calculation in chat
Calculator (views.py)	Tax rules (JSON/DB)	DB query	Get rule values
Admin (views.py)	Tax rules (models)	Django ORM	CRUD on rules
All apps	users (models)	Django ORM	Auth, user profiles
All apps	PostgreSQL	Django ORM	All data storage
phase2/retriever.py	ChromaDB	Python client	Vector similarity search
phase2/retriever.py	OpenAI API	HTTP	Generate embeddings
chat/views.py	Anthropic API	HTTP (SSE)	AI response generation

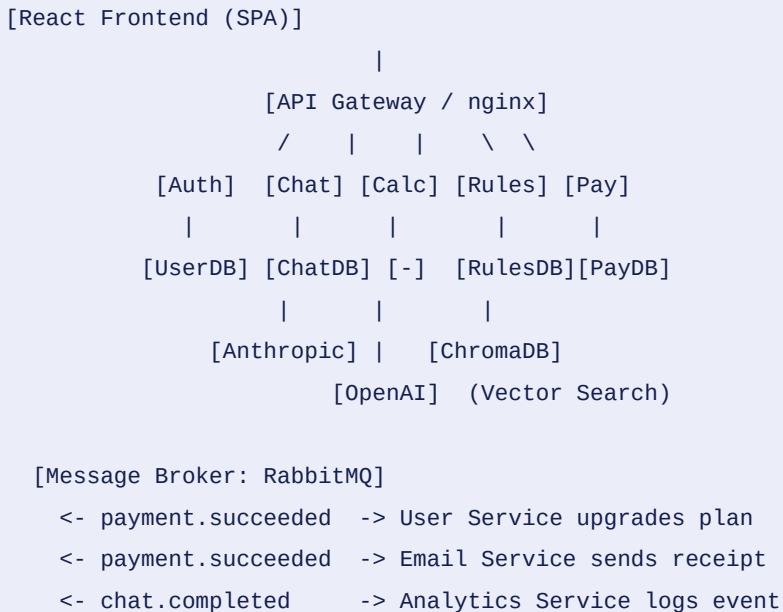
Natural Seams (Where to Split)

Looking at the dependency map above, the natural split points are clear. The following groups of functionality are relatively independent:

- * Auth + Users: everything about identity. Other services just need to verify a token.
- * Calculator: pure function -- input financial data, output tax numbers. No shared state.
- * Tax Rules + RAG: knowledge management and semantic search. No user state needed.
- * Chat: SSE streaming, conversation history. Depends on Auth + RAG + Calculator.
- * Payments: Stripe integration. Depends only on Auth to know who is paying.
- * Admin: staff-only management UI. Could be a backend-for-frontend, not a service.
- * Notifications/Email: fire-and-forget. Perfect for async events.

Section 9 -- TaxWijs as Microservices (Proposed Architecture)

The 7-Service Architecture



Data Flow: User Sends a Chat Message

1. Browser sends POST /api/chat/message with JWT in Authorization header
2. API Gateway verifies the JWT with Auth Service
3. Gateway routes the request to Chat Service
4. Chat Service calls Rules Service: 'retrieve relevant tax rules for this question'
5. Rules Service searches ChromaDB, returns top-5 relevant rules as text
6. Chat Service (if numbers needed) calls Calculator Service with the user's profile
7. Calculator Service computes tax figures and returns JSON
8. Chat Service assembles context (rules + numbers) + sends to Anthropic API
9. Anthropic streams the response back to Chat Service
10. Chat Service streams the SSE response back to the browser
11. (Async) Chat Service publishes 'chat.completed' event to RabbitMQ
12. (Async) Analytics Service consumes the event and logs usage

Section 10 -- The 7 Services: What Each Does

Service 1: Auth Service

Attribute	Detail
Responsibility	User registration, login, JWT issuance, token refresh, password reset
Database	PostgreSQL: users table, refresh tokens
API	POST /register, POST /token, POST /token/refresh, POST /password/reset
Exposes to	All other services (token verification endpoint)
Language	Python / Django (same as today, just isolated)
Size estimate	~500 lines of code

Service 2: Calculator Service

Attribute	Detail
Responsibility	Pure tax calculation. Input: financial profile. Output: tax breakdown JSON.
Database	None. Pure computation. Tax rule values fetched from Rules Service at startup.
API	POST /calculate (accepts income, user_type, deductions; returns full breakdown)
Exposes to	Chat Service, Admin UI, direct frontend calls
Language	Python or Rust (Rust would be 20x faster but Python is fine for this load)
Size estimate	~800 lines of code

Service 3: Tax Rules Service

Attribute	Detail
Responsibility	Manage tax rules CRUD, serve the RAG retrieval API, manage ChromaDB index
Database	PostgreSQL (rules metadata) + ChromaDB (vector embeddings)
API	GET /rules, POST /rules, GET /retrieve?q=question, POST /index/rebuild
Exposes to	Chat Service (retrieval), Calculator (rule values), Admin UI
Language	Python (ML dependencies: openai, chromadb, sentence-transformers)
Size estimate	~1,200 lines of code

Service 4: Chat Service

Attribute	Detail
Responsibility	Manage conversations, call Rules + Calculator, stream AI responses via SSE
Database	PostgreSQL: conversations, messages

API	POST /chat/message (SSE stream), GET /conversations, GET /conversations/:id
Exposes to	Frontend only
Calls	Auth Service (verify token), Rules Service (retrieve), Calculator (compute), Anthropic
Language	Python / Django (SSE streaming already works)
Size estimate	~600 lines of code

Service 5: Payments Service

Attribute	Detail
Responsibility	Stripe integration: create subscriptions, handle webhooks, manage billing
Database	PostgreSQL: subscriptions, invoices
API	POST /subscribe, POST /webhook, GET /subscription/status
Publishes	payment.succeeded, payment.failed, subscription.cancelled events to RabbitMQ
Language	Python or Node.js (both have excellent Stripe SDKs)
Size estimate	~400 lines of code

Service 6: User Profile Service

Attribute	Detail
Responsibility	Store user profiles, tax preferences, subscription plan, dashboard data
Database	PostgreSQL: profiles, settings, intake answers
API	GET /profile, PATCH /profile, GET /dashboard, GET /history
Subscribes to	payment.succeeded -> upgrades plan in DB
Language	Python / Django
Size estimate	~400 lines of code

Service 7: Notification Service

Attribute	Detail
Responsibility	Send emails (welcome, receipt, password reset, tax deadline reminders)
Database	None. Stateless.
Subscribes to	payment.succeeded, user.registered, password.reset events from RabbitMQ
Language	Python (with SendGrid or Mailgun SDK)
Size estimate	~200 lines of code

Section 11 -- Step-by-Step Migration Plan

The migration from monolith to microservices should follow the 'Strangler Fig' pattern: you extract one service at a time, redirect traffic to it, verify it works, then extract the next. You never do a 'big bang' rewrite -- that is how projects fail.

[TIP] The Strangler Fig pattern: Named after a vine that grows around a tree, gradually replacing it. You wrap the monolith in an API Gateway, then extract services one by one. The monolith handles requests until each extracted service is ready.

Phase 0: Preparation -- Before Extracting Anything [2-3 weeks]

- * Add an API Gateway (nginx or Traefik) in front of the monolith
- * Ensure every endpoint is covered by integration tests
- * Set up centralized logging (ELK stack or Grafana Loki)
- * Set up distributed tracing (Jaeger -- open source, free)
- * Create a Docker Swarm cluster (3 VPS nodes minimum)
- * Define API contracts for all services (OpenAPI/Swagger specs)
- * Set up CI/CD pipeline for each planned service repo

Phase 1: Extract Auth Service [2-3 weeks]

- * Create new standalone Django app: auth-service/
- * Copy users/models.py (User model, RefreshToken) and migrate its own PostgreSQL database
- * Implement /register, /token, /token/refresh, /password/reset endpoints
- * Add token verification endpoint: POST /verify -> returns user_id, is_admin
- * Configure API Gateway to route /api/users/token/* to new auth-service
- * Update all other monolith views: call auth-service.verify() instead of local JWT check
- * Run all integration tests. If passing, delete auth code from monolith.
- * Monitor for 2 weeks before proceeding.

Phase 2: Extract Calculator Service [2-3 weeks]

- * Create calculator-service/ as a standalone Python app (FastAPI or Django)
- * Move all calculator logic into the new service
- * At startup, fetch rule values from monolith's /api/tax/rules/ (until Rules service exists)
- * Expose: POST /calculate with same request/response schema as current endpoint
- * Route /api/calculator/ in API Gateway to new calculator-service
- * Remove calculator app from monolith
- * Test: calculator results must match monolith results for all 6 scenarios

Phase 3: Extract Tax Rules + RAG Service [3-4 weeks]

- * Create rules-service/ with its own PostgreSQL + ChromaDB
- * Migrate tax rules data from monolith database to rules-service database
- * Migrate phase2/ RAG code into the rules-service
- * Expose: GET /rules, POST /rules, GET /retrieve?q=..., POST /admin/rebuild-index
- * Update Calculator service to call rules-service instead of monolith
- * Update Chat (still in monolith) to call rules-service for RAG retrieval
- * Route /api/tax/* in API Gateway to rules-service
- * Run accuracy tests: precision@5 must still be >=95%

Phase 4: Extract Chat Service [3-4 weeks]

- * Create chat-service/ with its own PostgreSQL for conversations
- * Migrate conversation + message data from monolith
- * Implement SSE streaming, calling rules-service and calculator-service internally
- * This is the most complex extraction -- chat depends on both Rules and Calculator
- * Test SSE streaming thoroughly: Persian text, long responses, connection drops
- * Route /api/chat/* in API Gateway to chat-service
- * Remove chat app from monolith

Phase 5: Extract Payments + Notifications [2-3 weeks]

- * Create payments-service/ with Stripe SDK
- * Set up RabbitMQ broker (or AWS SQS as managed alternative)
- * Payments service publishes payment.succeeded events
- * Create notification-service/ that subscribes and sends emails
- * Create user-profile-service/ that subscribes and upgrades plan
- * Route /api/payments/* to payments-service
- * Test the full subscription flow end-to-end
- * Monitor for 2 weeks

Phase 6: Decommission Monolith [1-2 weeks]

- * Verify all routes are handled by dedicated services
- * Run full integration test suite against the microservices
- * Keep monolith running in read-only mode for 2 weeks as fallback
- * If no issues: shut down the monolith
- * Archive the monolith codebase (do not delete -- it is valuable reference)
- * Update documentation and onboarding guides
- * Celebrate -- the migration is complete

Total Time Estimate

Phase	Duration	Cumulative
Phase 0: Preparation	2-3 weeks	Month 1
Phase 1: Auth Service	2-3 weeks	Month 1-2
Phase 2: Calculator Service	2-3 weeks	Month 2
Phase 3: Rules + RAG Service	3-4 weeks	Month 2-3
Phase 4: Chat Service	3-4 weeks	Month 3-4
Phase 5: Payments + Notifications	2-3 weeks	Month 4-5
Phase 6: Decommission Monolith	1-2 weeks	Month 5
TOTAL	5-6 months	Full migration complete

[INFO] This is the estimate for a single experienced developer working part-time on the migration while keeping the monolith running. With a dedicated team of 2-3, it could be done in 2-3 months.

Section 12 -- Infrastructure You Would Need

Minimum Infrastructure for TaxWijs Microservices

Component	Purpose	Tool / Option	Monthly Cost (est.)
3x VPS (4GB RAM each)	Docker Swarm cluster nodes	Hetzner CX22	EUR 18/mo
API Gateway	Route traffic, auth, rate limit	nginx or Traefik	Free (on your VPS)
Message Broker	Async events between	RabbitMQ (Docker)	Free (on your VPS)
Centralized Logging	All service logs in one place	Grafana Loki + Grafana	Free (on your VPS)
Distributed Tracing	Trace requests across	Jaeger	Free (on your VPS)
Container Registry	Store Docker images	GitHub Container Registry	Free
CI/CD	Auto-deploy on git push	GitHub Actions	Free
SSL	HTTPS certificates	Let's Encrypt	Free
Monitoring	Uptime, performance alerts	Grafana + Prometheus	Free (on your VPS)

Approximate additional monthly cost vs current setup: EUR 12-18/month for extra VPS nodes.

7 Separate Databases

Each service needs its own database. You have two options:

- * Option A (Simpler): One PostgreSQL server with 7 databases -- each service connects to its own database on the shared server. Cheaper but one PostgreSQL crash affects all services.
- * Option B (True isolation): One PostgreSQL container per service -- each service runs its own DB container. More isolation but 7 DB processes running simultaneously.

Recommendation: Start with Option A (one server, 7 databases) and move to Option B as you grow.

New .env Structure (One Per Service)

```
auth-service/.env:
  DATABASE_URL=postgres://auth_user:pass@db:5432/auth_db
  SECRET_KEY=...
  JWT_SECRET=...

calculator-service/.env:
  RULES_SERVICE_URL=http://rules-service:8003

chat-service/.env:
  ANTHROPIC_API_KEY=...
  RULES_SERVICE_URL=http://rules-service:8003
```



```
CALCULATOR_SERVICE_URL=http://calculator-service:8002
AUTH_SERVICE_URL=http://auth-service:8001
DATABASE_URL=postgres://chat_user:pass@db:5432/chat_db
```

payments-service/.env:

```
STRIPE_SECRET_KEY=...
RABBITMQ_URL=amqp://user:pass@rabbitmq:5672/
```

Section 13 -- The Honest Recommendation

You have read everything about microservices: the benefits, the costs, the concepts, and exactly how to migrate TaxWijs. Now here is the honest recommendation.

Should TaxWijs Migrate to Microservices RIGHT NOW?

[WARN] NO. Not yet. The monolith is the right architecture for TaxWijs at its current stage. Here is why -- and what you should do instead.

Why the Monolith Is the Right Choice Now

- * Team size: Microservices are designed for multiple teams working independently. With 1-2 developers, the coordination overhead of microservices hurts more than it helps.
- * Traffic: TaxWijs does not have 10x traffic spikes that require independent scaling yet. The monolith can handle tens of thousands of requests per day on a EUR 6/mo VPS.
- * Feature velocity: A monolith is significantly faster to iterate on. Adding a new field to the user profile in a monolith is 1 migration. In microservices, it may require changes to 2-3 services and their API contracts.
- * Reliability: The current 3-container setup has 3 things that can fail. Microservices have 25+ things that can fail.
- * Cost: EUR 6/month vs EUR 50+/month for a real microservices setup.

What to Do Instead: The 'Modular Monolith'

The best path for TaxWijs right now is to make the monolith 'microservices-ready' without actually splitting it. This is called a 'Modular Monolith' or 'Majestic Monolith'. It gives you 80% of the architectural benefits with 5% of the operational overhead.

- * Keep clear module boundaries inside Django: users, chat, calculator, tax, payments
- * Each module should only communicate with others through defined service classes (not ORM imports)
- * Write API contracts (OpenAPI specs) for each module even though they are internal
- * Use signals or an internal event bus for cross-module events (instead of direct calls)
- * Keep separate test suites per module
- * When you need to extract a service later, the interface is already clean and well-tested

When IS the Right Time to Migrate?

Signal	What It Means	Which Service to Extract First
--------	---------------	--------------------------------

Calculator is slow under load	Too many simultaneous	Extract Calculator Service
New developers need to	Codebase too large	Extract the most complex module first
You want to rewrite Calculator in	Technology mismatch	Extract Calculator, rewrite in Rust/Go
Hiring a second team to work on	Team scaling	Extract Payments Service
Tax rules need to be updated without redeploying the whole app	Release independence needed	Extract Tax Rules Service

The Smart Path Forward

1. TODAY: Deploy the current monolith. Get real users. Measure what is slow.
2. MONTH 3-6: Build the Modular Monolith (clean internal boundaries, no cross-module ORM).
3. MONTH 6-12: If traffic or team grows, extract the first service (Calculator is lowest risk).
4. YEAR 2+: Extract services based on real bottlenecks, not theory.

[OK] The companies that successfully migrated to microservices (Netflix, Uber, Airbnb) all started as monoliths and extracted services as specific problems emerged. They did not start with microservices. They earned them.

Appendix A -- Microservices Glossary

API Gateway

A reverse proxy that sits in front of all microservices. Routes requests, handles auth, rate limiting, and SSL. Single entry point for all client traffic.

Circuit Breaker

A pattern that stops calling a failing service and returns a fallback response, preventing cascade failures.

Container Orchestration

A system that manages starting, stopping, scaling, and healing containers. Kubernetes and Docker Swarm are examples.

Database per Service

The principle that each microservice owns its own database. No service may directly read another service's database.

Docker Swarm

Docker's built-in orchestration tool. Simpler than Kubernetes. Good for 1-10 servers and small teams.

Event-Driven Architecture

Services communicate by publishing and subscribing to events via a message broker rather than calling each other directly.

gRPC

A high-performance RPC framework using Protocol Buffers. Faster than REST but more complex to set up.

Idempotency

An operation that produces the same result whether called once or many times. Critical for retry logic in distributed systems.

Jaeger

An open-source distributed tracing system. Shows you the path a request took across multiple services.

Kafka

A distributed event streaming platform. Handles millions of events per second. More powerful than RabbitMQ but more complex.

Kubernetes (K8s)

The industry-standard container orchestration platform. Self-heals, auto-scales, and manages containers across many servers.

Message Broker

Middleware that receives messages from producers and delivers them to consumers. RabbitMQ and Kafka are message brokers.

Monolith

An application where all functionality runs in one deployable unit. The opposite of microservices.

Modular Monolith

A monolith with clear internal boundaries that prevent modules from coupling. Often the right intermediate step

before full microservices.

RabbitMQ

A message broker implementing the AMQP protocol. Good for task queues and routing events between services.

Saga Pattern

A pattern for managing distributed transactions across multiple services using a sequence of local transactions with compensating actions on failure.

Service Discovery

The mechanism by which services find each other's network addresses. Kubernetes does this automatically via DNS.

Service Mesh

Infrastructure layer (Istio, Linkerd) that adds traffic management, mTLS encryption, and observability to all inter-service communication.

Sidecar Pattern

A separate container deployed alongside the main service container, handling cross-cutting concerns like logging, tracing, and proxying.

Strangler Fig Pattern

A migration strategy where you gradually extract services from a monolith one at a time, routing traffic to new services as they are ready.